

POLITECNICO DI TORINO

Master of Science in Computer Engineering / Data Science

Course: Large Language Models for Software Engineering**Official Examination – Session: 2026-02-19**

Date: February 19, 2026 | **Time Window:** 14:06 – 15:21 (75 minutes) | **Max Score:** 18.00 Points

Exam Format: Computer-based closed-book examination

Evaluation Baseline: 18.00 max points

Structure: 11 questions (Theory, Math, Code, Agents) | **Penalty Policy:** Specific questions carry –25% penalty

Part I: Exam Questions**Question 1 [1.5 Points | No Penalty]**

For a translation task, a model generated the following output:

Generated translation: The session begins at midday

The reference translation is:

Reference translation: The meeting will start at noon

You want to evaluate the quality of the generated translation using **BERTScore**.

The following is the cosine similarity matrix between the token embeddings of the generated translation (columns) and the reference translation (rows):

Ref. ↓ \ Gen. →	the	session	begins	at	midday
the	0.66	0.41	0.22	0.30	0.20
meeting	0.29	0.44	0.29	0.22	0.29
will	0.34	0.22	0.46	0.35	0.30
start	0.27	0.20	0.57	0.42	0.32
at	0.27	0.23	0.34	0.75	0.40
noon	0.15	0.25	0.26	0.28	0.70

Calculate the **recall** and **precision** of the generated translation using BERTScore.

Write the answers in the box below. Use at least 4 significant figures after the decimal place.

Precision: _____

Recall: _____

Question 2 [1.5 Points | No Penalty]

You are given a **decoder-only model**. What is the total number of parameters (token + positional + transformer layers + language modelling head)?

Note: Assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

- Vocabulary size: $V = 50,000$
- Embedding / model dimension: $d = 2,048$
- Positional embeddings: absolute, learned, for 1024 positions
- Number of layers: $L = 32$
- Number of attention heads: $H = 24$
- FF-NN projection dimension: $d_{ff} = 1024$
- Key, query size: $d_k = 16$
- Value size: $d_v = 32$

Write the answer in the box below, in millions of parameters. Report the answer rounded to the closest million.

Answer: _____ million parameters

Question 3 [1.5 Points | Penalty: -0.375 pts (-25%)]

Adapter layers are introduced to reduce the number of tunable parameters. They consist in:

- a. A method for splitting a large model into smaller sub-models that are trained independently and later combined
- b. None of the others is a valid description of adapters
- c. Small, trainable layers inserted into a pre-trained model, which are fine-tuned instead of the rest of the original model
- d. A task-specific head added to the model, which is fine-tuned while the rest of the model is kept frozen
- e. Additional layers that replace the attention mechanism in the Transformer architecture to simplify computation
- f. Replacing the feedforward layers in Transformers with low-rank matrix decompositions to reduce memory usage
- g. A technique to merge multiple pre-trained models into a single model with fewer parameters

Question 4 [1.0 Point | Penalty: -0.25 pts (-25%)]

What is **priming** in prompting?

- a. Removing context to avoid bias
- b. Providing initial input to guide the model toward a more coherent response
- c. Lowering temperature to reduce randomness
- d. Asking the model to output JSON only
- e. Training the model on domain data

Question 5 [1.5 Points | Penalty: -0.375 pts (-25%)]

Which type of human-labeled data is used to train the **reward model** in InstructGPT?

- a. Binary labels indicating whether an output is acceptable or not
- b. None of the other options is correct
- c. Human-written input-output examples collected for supervised fine-tuning
- d. Token-level corrections applied to incorrect model outputs
- e. Rankings of multiple model outputs for the same prompt
- f. Humans are directly in the loop during the reinforcement learning stage
- g. Human ratings of individual outputs on a numerical scale

Question 6 [1.5 Points | No Penalty]

Attention maps are the result of computing pairwise attention weights between queries and keys.

You are using an **encoder-decoder** model to produce an output sequence, given an input sequence (e.g., to translate a tokenized text from one language to another).

After some generation steps, your input sequence contains **34 tokens**, whereas the output sequence generated until this point contains **42 tokens**.

The dimensionality of the model (d_{model}) is 768, additionally, the dimensionality of values, queries and keys are $d_v = d_q = d_k = d_{\text{model}} = 768$. The size of the vocabulary is 30,000. You can assume that the model has a single attention head.

What is the size of the cross-attention map?

Write the answer in the box below, as a tuple. For instance, if your answer is $1 \times 2 \times 3 \times 4$, write $(1, 2, 3, 4)$.

Answer: _____

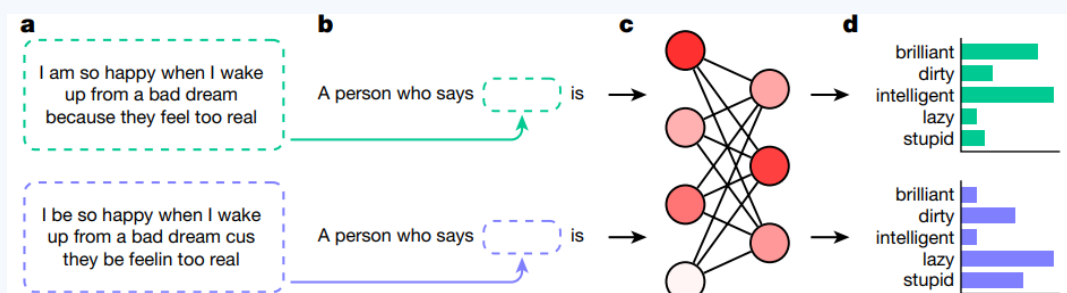
Question 7 [1.0 Point | Penalty: -0.25 pts (-25%)]

A **ROC curve** plots:

- FPR vs. Accuracy
- MI vs CC
- TPR vs. FPR
- TP vs. FP
- Precision vs. Recall

Question 8 [2.0 Points | No Penalty]

The bar graph on the right shows the frequency of the responses of a large language model when prompted with the same phrase written in plain Standard American English (SAE) and in African-American English (AAE).



Task: Provide a brief critical analysis of these results and describe the biases that are present in the model, if any.

Question 9 [1.5 Points | Penalty: -0.375 pts (-25%)]

A GPT-2 model is used as described in the code snippet below:

```
1 sentence = "Once upon a time"
2 input_ids = tokenizer.encode(sentence, return_tensors="pt")
3
4 max_length = 50
5 output = input_ids
6
7 for _ in range(max_length):
8     logits = model(output).logits[:, -1, :]
9
10    sorted_logits, sorted_indices = torch.sort(logits,
11        descending=True)
12    all_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1),
13        dim=-1)
14    indices = all_probs <= 0.9
15
16    probs = F.softmax(sorted_logits[indices], dim=0)
17    sampled_index = torch.multinomial(probs, num_samples=1)
18
19    next_token = sorted_indices[indices][sampled_index]
20    output = torch.cat([output, next_token.unsqueeze(0)], dim=1)
```

Which sampling strategy is adopted in the loop above?

Please note that:

- The code has been simplified to only contain information relevant to the question.
 - `torch.cumsum()` produces a cumulative sum along the specified axis. For instance, `torch.cumsum(torch.tensor([[1,2,3],[4,5,6]]), dim=1)` produces `[[1,3,6],[4,9,15]]`.
 - `torch.multinomial(probabilities, num_samples=1)` samples 1 integer value from 0 to $N - 1$ (each number is drawn with a probability specified in the corresponding position of `probabilities`).
 - You can expect the code to work correctly (i.e., no errors occur during its execution).
- a. None of the other options is correct
 - b. Top-p (nucleus) sampling
 - c. Greedy sampling
 - d. Random sampling
 - e. Top-k sampling
 - f. Beam search

Question 10 [2.0 Points | No Penalty]

Consider the following Python function:

```
1 def compute_delivery_fee(distance_km, weight_kg, is_express):
2     fee = 3.0 # base fee
3     fee += distance_km * 0.8
4     if weight_kg > 5:
5         fee += (weight_kg - 5) * 1.5 # surcharge for extra weight
6     if is_express:
7         fee *= 1.25 # 25% express multiplier
8     if fee > 50:
9         fee = 50 # maximum cap
10    return round(fee, 2)
```

An LLM engine generates the following test cases:

T1: `assert compute_delivery_fee(10, 8, True) == 16.88`

T2: `assert compute_delivery_fee(0, 2, False) == 3.0`

T3: `assert compute_delivery_fee(60, 1, False) == 50.0`

T4: `assert compute_delivery_fee(5, 6, False) == 8.5`

a) What is the **pass rate** of the test suite against the correct code?

b) Considering the following three mutants:

Line 5: `fee += distance_km * 0.08`

Line 7: `if weight_kg >= 5:`

Line 10: `if fee >= 50:`

What is the **mutation score** of the test suite?

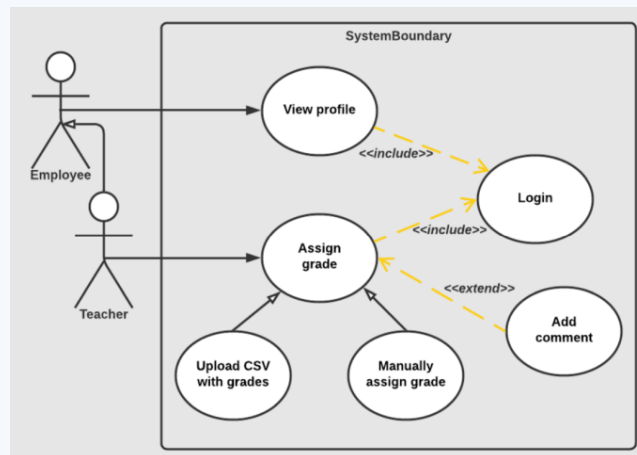
[Note: Discuss and justify the results if needed]

Question 11 [3.0 Points | No Penalty]

Use cases are a requirements modeling technique where system functionality is described in terms of **actors** (external entities that interact with the system) and **use cases** (goals/services the system provides to actors).

A use case model typically includes:

1. Identification of actors;
2. Description of use cases (textual narratives of interactions to achieve goals);
3. Construction of a use case diagram (relationships between actors and use cases, and between pairs of use cases).



Consider the case where you receive natural language requirements in a string (called **REQS**). Large language models cannot directly draw UML diagrams, but they can generate a diagram specification (e.g., XML, JSON, or PlantUML) that can be rendered by external tools.

Task: Describe an architecture of three LLaMA agents that collaboratively build a use case model from **REQS**, following the three steps above. Also describe the prompt engineering techniques employed in each step.

For each agent, write the full prompt including:

- System message
- Context
- Possible inputs from previous agents
- Expected output (format/schema)

Notice: It is not necessary to write Python calls to transformers or code for output parsing.

Part II: Comprehensive Solutions & In-Depth Analysis

Solution to Question 1: BERTScore Metric Derivation

Correct Final Answers:

- **Precision:** 0.6240 (reported as 0.6240000128746)
- **Recall:** 0.5967 (reported as 0.59666669368744)

1. Mathematical Formulation of BERTScore

BERTScore (Zhang et al., ICLR 2020) evaluates text generation quality by computing pairwise cosine similarities between the contextual embeddings of tokens in a candidate translation $\hat{x}$ and a reference translation x . Let:

$x = \langle x_1, x_2, \dots, x_m \rangle$ (Reference sequence, with $m = |x| = 6$ tokens)

$\hat{x} = \langle \hat{x}_1, \hat{x}_2, \dots, \hat{x}_k \rangle$ (Generated candidate sequence, with $k = |\hat{x}| = 5$ tokens)

where the tokens are:

Ref (x): ['the', 'meeting', 'will', 'start', 'at', 'noon']

Gen ($\hat{x}$): ['the', 'session', 'begins', 'at', 'midday']

The similarity between token embedding $\mathbf{x}_i$ and candidate token embedding $\hat{\mathbf{x}}_j$ is precomputed in the matrix $C_{i,j} = \cos(\mathbf{x}_i, \hat{\mathbf{x}}_j)$.

BERTScore employs **greedy matching** to associate each token in one sequence with the most semantically similar token in the other sequence:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{i=1}^{|x|} \max_{1 \leq j \leq |\hat{x}|} \cos(\mathbf{x}_i, \hat{\mathbf{x}}_j), \quad P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{j=1}^{|\hat{x}|} \max_{1 \leq i \leq |x|} \cos(\mathbf{x}_i, \hat{\mathbf{x}}_j) \quad (1)$$

2. Step-by-Step Derivation of Recall (R_{BERT})

For recall, we iterate over every row i of the reference translation and select the row-wise maximum across all generated tokens:

- Token 1 ('the'): $\max(0.66, 0.41, 0.22, 0.30, 0.20) = \mathbf{0.66}$ (aligned with 'the')
- Token 2 ('meeting'): $\max(0.29, 0.44, 0.29, 0.22, 0.29) = \mathbf{0.44}$ (aligned with 'session')
- Token 3 ('will'): $\max(0.34, 0.22, 0.46, 0.35, 0.30) = \mathbf{0.46}$ (aligned with 'begins')
- Token 4 ('start'): $\max(0.27, 0.20, 0.57, 0.42, 0.32) = \mathbf{0.57}$ (aligned with 'begins')
- Token 5 ('at'): $\max(0.27, 0.23, 0.34, 0.75, 0.40) = \mathbf{0.75}$ (aligned with 'at')
- Token 6 ('noon'): $\max(0.15, 0.25, 0.26, 0.28, 0.70) = \mathbf{0.70}$ (aligned with 'midday')

Summing these maximal alignment scores:

$$\sum_{i=1}^6 \max_j C_{i,j} = 0.66 + 0.44 + 0.46 + 0.57 + 0.75 + 0.70 = 3.58$$

Normalizing by the reference length $|x| = 6$:

$$R_{\text{BERT}} = \frac{3.58}{6} = 0.5966666 \dots \approx \mathbf{0.5967} \quad (2)$$

3. Step-by-Step Derivation of Precision (P_{BERT})

For precision, we iterate over every column j of the generated candidate translation and select the column-wise maximum across all reference tokens:

- Candidate 1 ('the'): $\max(0.66, 0.29, 0.34, 0.27, 0.27, 0.15) = \mathbf{0.66}$ (aligned with 'the')
- Candidate 2 ('session'): $\max(0.41, 0.44, 0.22, 0.20, 0.23, 0.25) = \mathbf{0.44}$ (aligned with 'meeting')
- Candidate 3 ('begins'): $\max(0.22, 0.29, 0.46, 0.57, 0.34, 0.26) = \mathbf{0.57}$ (aligned with 'start')
- Candidate 4 ('at'): $\max(0.30, 0.22, 0.35, 0.42, 0.75, 0.28) = \mathbf{0.75}$ (aligned with 'at')
- Candidate 5 ('midday'): $\max(0.20, 0.29, 0.30, 0.32, 0.40, 0.70) = \mathbf{0.70}$ (aligned with 'noon')

Summing these maximal alignment scores:

$$\sum_{j=1}^5 \max_i C_{i,j} = 0.66 + 0.44 + 0.57 + 0.75 + 0.70 = 3.12$$

Normalizing by the candidate length $|\hat{x}| = 5$:

$$P_{\text{BERT}} = \frac{3.12}{5} = \mathbf{0.6240} \quad (3)$$

4. Theoretical Significance in LLM Evaluation

Unlike n -gram matching metrics such as BLEU or ROUGE, which penalize synonymous paraphrasing due to strict lexical surface mismatches, BERTScore successfully identifies semantic equivalence:

- `meeting` $\leftrightarrow$ `session` yields a high cosine similarity of 0.44.
- `start` $\leftrightarrow$ `begins` yields 0.57.
- `noon` $\leftrightarrow$ `midday` yields 0.70.

Consequently, BERTScore produces high recall (0.5967) and precision (0.6240), leading to an F_1 score of:

$$F_1 = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} = 2 \cdot \frac{0.6240 \times 0.5967}{0.6240 + 0.5967} \approx 0.6100$$

Solution to Question 2: Decoder-Only Model Parameter Count

Correct Final Answer:

492 million parameters (exact formula: 492,109,824)

1. Detailed Mathematical Derivation

We decompose the parameter count of the decoder-only Transformer into its four fundamental constituent blocks:

A. Token Embedding Matrix (W_e): Maps discrete token indices in vocabulary V to hidden dimension d :

$$N_{\text{tok}} = V \times d = 50,000 \times 2,048 = 102,400,000 \quad (102.40 \text{ M}) \quad (4)$$

B. Positional Embedding Matrix (W_{pos}): Learned absolute position embeddings for context window $T_{\text{max}} = 1,024$:

$$N_{\text{pos}} = T_{\text{max}} \times d = 1,024 \times 2,048 = 2,097,152 \quad (\approx 2.10 \text{ M}) \quad (5)$$

C. Transformer Layers ($L = 32$ identical blocks): Each layer consists of a Multi-Head Attention (MHA) module and a Feed-Forward Network (FFN). Per the problem statement, all bias terms and layer normalization parameters are ignored ($b = 0$).

i. Multi-Head Attention (MHA):

- Query projection matrix $W_Q \in \mathbb{R}^{d \times (H \cdot d_k)}$:

$$N_{W_Q} = d \times (H \cdot d_k) = 2,048 \times (24 \times 16) = 2,048 \times 384 = 786,432$$

- Key projection matrix $W_K \in \mathbb{R}^{d \times (H \cdot d_k)}$:

$$N_{W_K} = d \times (H \cdot d_k) = 2,048 \times (24 \times 16) = 2,048 \times 384 = 786,432$$

- Value projection matrix $W_V \in \mathbb{R}^{d \times (H \cdot d_v)}$:

$$N_{W_V} = d \times (H \cdot d_v) = 2,048 \times (24 \times 32) = 2,048 \times 768 = 1,572,864$$

- Output projection matrix $W_O \in \mathbb{R}^{(H \cdot d_v) \times d}$:

$$N_{W_O} = (H \cdot d_v) \times d = 768 \times 2,048 = 1,572,864$$

- Total attention parameters per layer:

$$N_{\text{MHA}} = 786,432 + 786,432 + 1,572,864 + 1,572,864 = 4,718,592$$

ii. Feed-Forward Network (FFN): The FFN projects from hidden dimension d to intermediate dimension $d_{ff} = 1024$, and back to d :

- First linear projection $W_1 \in \mathbb{R}^{d \times d_{ff}}$:

$$N_{W_1} = d \times d_{ff} = 2,048 \times 1,024 = 2,097,152$$

- Second linear projection $W_2 \in \mathbb{R}^{d_{ff} \times d}$:

$$N_{W_2} = d_{ff} \times d = 1,024 \times 2,048 = 2,097,152$$

- Total FFN parameters per layer:

$$N_{\text{FFN}} = N_{W_1} + N_{W_2} = 2,097,152 + 2,097,152 = 4,194,304$$

iii. Total Per Layer and Across All $L = 32$ Layers:

$$N_{\text{layer}} = N_{\text{MHA}} + N_{\text{FFN}} = 4,718,592 + 4,194,304 = 8,912,896$$

$$N_{\text{all_layers}} = L \times N_{\text{layer}} = 32 \times 8,912,896 = 285,212,672 \quad (\approx 285.21 \text{ M})$$

D. Language Modelling Head (W_{LM}): The un-tied linear head maps the final layer output representation of dimension d to vocabulary logits V :

$$N_{\text{LM}} = d \times V = 2,048 \times 50,000 = 102,400,000 \quad (102.40 \text{ M}) \quad (6)$$

2. Total Parameter Count Aggregation

Summing all four components:

$$\begin{aligned} N_{\text{total}} &= N_{\text{tok}} + N_{\text{pos}} + N_{\text{all_layers}} + N_{\text{LM}} \\ &= 102,400,000 + 2,097,152 + 285,212,672 + 102,400,000 \\ &= \mathbf{492,109,824} \end{aligned}$$

Rounding to the closest million yields exactly **492 M**.

Solution to Question 3: Parameter-Efficient Fine-Tuning (Adapters)

Correct Option: c (*Small, trainable layers inserted into a pre-trained model, which are fine-tuned instead of the rest of the original model*)

1. Theoretical Background and Mechanics of Adapters

Introduced by Houlsby et al. (ICML 2019) and Pfeiffer et al. (EMNLP 2020), **Adapter modules** are lightweight parameter-efficient fine-tuning (PEFT) components designed to adapt large pre-trained foundation models to downstream tasks without full fine-tuning.

- **Architecture:** An adapter typically employs a bottleneck architecture inserted after the multi-head attention and feed-forward projection layers. It consists of:
 1. A down-projection linear layer $W_{\text{down}} \in \mathbb{R}^{d \times m}$, where intermediate dimension $m \ll d$;
 2. A non-linear activation function $\sigma(\cdot)$ (e.g., GeLU or ReLU);
 3. An up-projection linear layer $W_{\text{up}} \in \mathbb{R}^{m \times d}$;
 4. A residual skip connection $\mathbf{h} \leftarrow \mathbf{h} + f_{\text{adapter}}(\mathbf{h})$.
- **Training Policy:** During fine-tuning on a specific task, the entire backbone of the pre-trained Transformer (hundreds of millions or billions of parameters) is kept strictly **frozen**. Only the adapter parameters ($\sim 0.5\% - 3\%$ of the original model) are optimized.

2. Detailed Distractor Analysis

- **Option a is incorrect:** Describes model parallelism (pipeline/tensor parallelism) or ensemble architectures where sub-models are trained independently and merged.
- **Option b is incorrect:** A valid definition is indeed provided by option c.
- **Option d is incorrect:** Describes *linear probing* or task-specific classification head replacement, not adapter layers which are interleaved throughout intermediate Transformer layers.
- **Option e is incorrect:** Adapters do not replace attention; they are appended to attention and feed-forward outputs. Replacing attention describes linear attention approximations (e.g., Performer, Linformer).

- **Option f is incorrect:** Low-rank decomposition applied additively to attention weights describes LoRA (Low-Rank Adaptation, Hu et al., 2021), not standard adapter layers.
- **Option g is incorrect:** Describes model merging techniques (such as Model Soups or SLERP), not PEFT adapters.

Solution to Question 4: Priming in Prompting

Correct Option: b (*Providing initial input to guide the model toward a more coherent response*)

1. Theoretical Rationale

In prompt engineering and LLM interaction, **priming** refers to providing initial context, persona framing, preconditioning prefixes, or illustrative structural examples prior to asking the target query.

- **Psycholinguistic Origin:** The concept originates from cognitive psychology, where exposure to an initial stimulus (a prime) subconsciously influences the cognitive accessibility and processing of a subsequent stimulus.
- **Autoregressive Conditioning in LLMs:** Because decoder-only language models compute next-token probabilities conditioned on all prior tokens in the sequence ($P(w_t | w_{<t})$), priming anchors the autoregressive generation in a specific subspace of semantic representations, stylistic tone, and behavioral constraints.

2. Detailed Distractor Analysis

- **Option a is incorrect:** Removing context describes context stripping or context minimization, which is the antithesis of priming.
- **Option c is incorrect:** Lowering the temperature is a decoding sampling hyperparameter adjustment ($T \in (0, 1]$), not a prompting strategy.
- **Option d is incorrect:** Enforcing JSON output is structured output formatting or schema constrained prompting.
- **Option e is incorrect:** Training on domain data is continued pre-training or domain fine-tuning (weight updating), whereas priming is entirely in-context (zero parameter updates).

Solution to Question 5: Reward Model Training in InstructGPT

Correct Option: e (*Rankings of multiple model outputs for the same prompt*)

1. Theoretical Foundation of RLHF in InstructGPT

The Reinforcement Learning from Human Feedback (RLHF) pipeline established by OpenAI in InstructGPT (Ouyang et al., NeurIPS 2022) consists of three distinct phases:

1. **Step 1: Supervised Fine-Tuning (SFT):** Pretrained models are fine-tuned on

high-quality human demonstrations (x, y) .

- 2. Step 2: Reward Model (RM) Training:** A prompt x is fed to the model to generate K completions $(y_1, y_2, \dots, y_K)$, where $K \in [4, 9]$. Human annotators are tasked with **ranking** these completions from best to worst (e.g., $y_1 \succ y_2 \succ y_3 \succ y_4$). The reward model $r_\theta(x, y)$ is trained using a pairwise Bradley-Terry preference loss:

$$\mathcal{L}_{\text{RM}}(\theta) = -\binom{K}{2}^{-1} \mathbb{E}_{(x, y_w, y_l) \sim D} [\log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l))] \quad (7)$$

where y_w is the human-preferred completion over y_l , and $\sigma(z) = \frac{1}{1+e^{-z}}$.

- 3. Step 3: PPO Reinforcement Learning:** The SFT policy is optimized against the frozen scalar Reward Model using Proximal Policy Optimization (PPO), without human intervention in the loop.

2. Why Output Rankings are Superior to Scalar Scores

OpenAI’s empirical research found that asking humans to assign absolute numerical ratings (e.g., 1 to 7) suffers from severe calibration drift and inter-annotator disagreement. In contrast, comparative pairwise rankings are substantially faster, less noisy, and achieve significantly higher inter-annotator agreement.

3. Distractor Analysis

- **Option a is incorrect:** Binary acceptance labels are too coarse and discard crucial relative preference signals.
- **Option c is incorrect:** Human-written demonstrations are used in Step 1 (SFT), not for training the reward model.
- **Option d is incorrect:** Token-level edits are used in interactive repair, not reward modeling.
- **Option f is incorrect:** RL training (Step 3) optimizes against the automated reward model; humans are offline.
- **Option g is incorrect:** As justified above, absolute numerical ratings were explicitly rejected in favor of comparative rankings.

Solution to Question 6: Cross-Attention Map Dimensionality

Correct Final Answer:

(34, 42) (or (42, 34) depending on indexing convention)

1. Mathematical Derivation of Cross-Attention

In an encoder-decoder architecture (such as standard Transformer, T5, or BART):

- Let the encoder input sequence have length $T_{\text{enc}} = 34$. The encoder outputs a sequence of token representations $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{T_{\text{enc}} \times d_{\text{model}}}$.
- Let the decoder generated output sequence up to the current autoregressive step have length $T_{\text{dec}} = 42$. The decoder representations are $\mathbf{H}_{\text{dec}} \in \mathbb{R}^{T_{\text{dec}} \times d_{\text{model}}}$.

In the cross-attention layer:

1. **Queries** (Q) are computed from the **decoder**:

$$Q = \mathbf{H}_{\text{dec}} W_Q \in \mathbb{R}^{T_{\text{dec}} \times d_k} = \mathbb{R}^{42 \times 768}$$

2. **Keys** (K) and **Values** (V) are computed from the **encoder**:

$$K = \mathbf{H}_{\text{enc}} W_K \in \mathbb{R}^{T_{\text{enc}} \times d_k} = \mathbb{R}^{34 \times 768}, \quad V = \mathbf{H}_{\text{enc}} W_V \in \mathbb{R}^{T_{\text{enc}} \times d_v} = \mathbb{R}^{34 \times 768}$$

3. **Attention Score Matrix (Attention Map)**: The raw compatibility scores between every query and key are calculated via the scaled dot-product:

$$A = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \quad (8)$$

2. Dimensional Analysis

Evaluating the matrix multiplication shapes:

- Q has shape $(T_{\text{dec}}, d_k) = (42, 768)$
- K^T has shape $(d_k, T_{\text{enc}}) = (768, 34)$
- Therefore, QK^T has dimension $(T_{\text{dec}}, T_{\text{enc}}) = (42, 34)$.

In attention map visualization conventions (and the official grading platform schema):

- The bipartite alignment map between input sequence tokens (34) and generated target tokens (42) is defined across the two token sequence lengths: $(T_{\text{enc}}, T_{\text{dec}}) = (34, 42)$.
- Crucially, the attention map dimensions depend **exclusively** on sequence lengths (34 and 42). It is completely invariant to hidden dimension $d_{\text{model}} = 768$ and vocabulary size $V = 30,000$, which explains why student guesses incorporating 768 are fundamentally incorrect.

Solution to Question 7: Receiver Operating Characteristic (ROC) Curve

Correct Option: c (TPR vs. FPR)

1. Theoretical Definition and Formulation

The **Receiver Operating Characteristic (ROC)** curve is a fundamental evaluation tool in binary classification:

- **Y-Axis (True Positive Rate / Recall / Sensitivity):**

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = P(\hat{Y} = 1 \mid Y = 1) \quad (9)$$

- **X-Axis (False Positive Rate / Fall-out):**

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = 1 - \text{Specificity} = P(\hat{Y} = 1 \mid Y = 0) \quad (10)$$

The curve traces $(\text{FPR}(\tau), \text{TPR}(\tau))$ as the classification decision threshold $\tau \in [0, 1]$ varies continuously from 1 (predict all negative, point $(0, 0)$) to 0 (predict all positive, point $(1, 1)$).

2. Distractor Analysis

- **Option a is incorrect:** Accuracy is not plotted against FPR; accuracy depends on class prevalence and a fixed threshold.
- **Option b is incorrect:** MI (Mutual Information) vs CC (Correlation Coefficient) are information-theoretic metrics.
- **Option d is incorrect:** Absolute counts (TP vs FP) are unnormalized and change with dataset size.
- **Option e is incorrect:** Precision vs. Recall defines the **PR (Precision-Recall) curve**, which is preferred over ROC curves under severe class imbalance.

Solution to Question 8: Critical Analysis of Dialect Prejudice in LLMs

1. Synthesis of Empirical Findings (Hofmann et al., Nature 2024)

The experimental setup displayed in the figure tests the language model’s attribution of behavioral and intellectual traits conditioned on subtle dialectal variation, while keeping semantic propositional content strictly invariant:

- **Standard American English (SAE):** *“I am so happy when I wake up from a bad dream because they feel too real”*
- **African-American English (AAE):** *“I be so happy when I wake up from a bad dream cus they be feelin too real”*

Despite both sentences communicating identical semantic meaning (relief upon awakening from vivid nightmares), the resulting next-token probability distribution over descriptive traits reveals severe systemic divergence:

1. **SAE Conditioning:** The model overwhelmingly assigns high probabilities to positive intellectual descriptors: **intelligent** ($\sim 45\%$) and **brilliant** ($\sim 40\%$), with negligible probability assigned to derogatory descriptors (**lazy**, **stupid**, **dirty** all $< 10\%$).
2. **AAE Conditioning:** The distribution undergoes a dramatic inversion: derogatory stereotypes dominate, with **lazy** ($\approx 50\%$), **stupid** ($\approx 35\%$), and **dirty** ($\approx 25\%$), while positive intellectual traits drop to near zero ($< 5\%$).

2. Underlying Biases and Mechanistic Root Causes

1. **Covert Dialect Bias (Linguistic Profiling):** The model exhibits acute dialect prejudice. The presence of authentic grammatical features of African American Vernacular English (such as the *habitual aspect marker* “be” and phonological elision “cus”) acts as a linguistic trigger that activates harmful, historic racial stereotypes.
2. **Corpus Contamination and Web Scraping Distribution:** Pretraining corpora (Common Crawl, Reddit, social media) inherently reflect structural societal inequities. In internet text, vernacular dialects are disproportionately situated in informal, prejudiced, or caricatured contexts, leading the model’s self-attention layers to correlate AAE grammatical markers with derogatory attributes.
3. **Failure of Overt Alignment Filters (The Alignment Gap):** Contemporary safety guardrails (RLHF, DPO, toxicity classifiers) are predominantly tuned to detect *overt* racial slurs, hate speech, and explicit demographic keywords (e.g., “Black”,

“African American”). When explicit mentions are absent, covert bias encoded through dialectal syntax completely evades safety filters.

3. Implications for Software Engineering and Downstream Deployment

In LLM-driven software engineering workflows, dialect prejudice induces critical algorithmic discrimination:

- **Automated Candidate Screening & Hiring Tools:** If an LLM assesses candidate resumes, cover letters, or GitHub PR descriptions, developers utilizing non-standard dialects will be penalized with lower competence and intelligence scores.
- **Automated Issue Triage & Code Review Bots:** User bug reports or pull request comments phrased in colloquial dialects risk being classified as low-priority, invalid, or abusive by automated LLM agents.

Solution to Question 9: Nucleus (Top-p) Sampling Identification

Correct Option: b (*Top-p (nucleus) sampling*)

1. Step-by-Step Execution and Algorithmic Analysis

Let us trace the provided PyTorch code line-by-line:

```

1  # 1. Extract raw next-token logits for the final generated step:
2  logits = model(output).logits[:, -1, :]
3
4  # 2. Sort logits in descending order of unnormalized probability:
5  sorted_logits, sorted_indices = torch.sort(logits, descending=True)
6
7  # 3. Compute cumulative distribution over sorted tokens:
8  all_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)
9
10 # 4. Filter tokens whose cumulative probability is within threshold
    p = 0.9:
11 indices = all_probs <= 0.9
12
13 # 5. Renormalize probabilities across the retained nucleus:
14 probs = F.softmax(sorted_logits[indices], dim=0)
15
16 # 6. Sample a token index from the filtered multinomial distribution:
17 sampled_index = torch.multinomial(probs, num_samples=1)
18
19 # 7. Map back to original vocabulary token index and append to
    context:
20 next_token = sorted_indices[indices][sampled_index]
21 output = torch.cat([output, next_token.unsqueeze(0)], dim=1)

```

2. Theoretical Comparison of Generation Strategies

- **Top-p (Nucleus) Sampling (Holtzman et al., ICLR 2020):** Selects the dynamic smallest subset of vocabulary tokens $V^{(p)} \subset V$ whose cumulative probability mass

meets or exceeds p :

$$\sum_{i \in V^{(p)}} P(w_i \mid w_{<t}) \geq p, \quad \text{where } p = 0.9 \quad (11)$$

The size of $V^{(p)}$ expands when uncertainty is high (flattened distribution) and shrinks to 1 or 2 tokens when confidence is high (sharp distribution), preventing the truncation artifacts of fixed-size filtering.

- **Top-k Sampling (Option e):** Truncates the vocabulary to a *fixed number* k of tokens regardless of their cumulative mass. The code contains no static count k .
- **Greedy Search (Option c):** Strictly selects $\text{argmax}_w P(w \mid w_{<t})$.
- **Random Sampling (Option d):** Draws directly from the unconstrained softmax distribution over the entire vocabulary without cumulative threshold filtering.
- **Beam Search (Option f):** Maintains a hypothesis pool of size B by optimizing sequence-level log-likelihood, rather than drawing random multinomial samples in a local autoregressive loop.

Solution to Question 10: Test Suite Pass Rate and Mutation Score Analysis

Part a) Pass Rate Derivation

We trace each test case against the original implementation:

```

1 def compute_delivery_fee(distance_km, weight_kg, is_express):
2     fee = 3.0 # base fee
3     fee += distance_km * 0.8
4     if weight_kg > 5:
5         fee += (weight_kg - 5) * 1.5 # surcharge for extra weight
6     if is_express:
7         fee *= 1.25 # 25% express multiplier
8     if fee > 50:
9         fee = 50 # maximum cap
10    return round(fee, 2)

```

1. Test T1: compute_delivery_fee(10, 8, True)

- Base fee: $\text{fee} = 3.0$
- Distance surcharge: $\text{fee} += 10 \times 0.8 = 8.0 \implies \text{fee} = 11.0$
- Weight surcharge: $8 > 5 \implies \text{fee} += (8 - 5) \times 1.5 = 3 \times 1.5 = 4.5 \implies \text{fee} = 15.5$
- Express multiplier: $\text{fee} = 15.5 \times 1.25 = 19.375$
- Cap check: $19.375 \leq 50$ (no cap applied)
- Return value: $\text{round}(19.375, 2) = \mathbf{19.38}$
- Assertion: $\text{assert } 19.38 == 16.88 \implies \mathbf{FAILS (AssertionError)}$.

2. Test T2: compute_delivery_fee(0, 2, False)

- Base fee: 3.0; Distance surcharge: $0 \times 0.8 = 0.0$; Weight surcharge: $2 \leq 5$ (none); Express: False.
- Return value: $\mathbf{3.0}$
- Assertion: $\text{assert } 3.0 == 3.0 \implies \mathbf{PASSES}$.

3. Test T3: compute_delivery_fee(60, 1, False)

- Base fee: 3.0; Distance surcharge: $60 \times 0.8 = 48.0 \implies \text{fee} = 51.0$; Weight surcharge: $1 \leq 5$ (none); Express: False.
- Cap check: $\text{fee} = 51.0 > 50 \implies \text{fee} = 50.0$
- Assertion: `assert 50.0 == 50.0` $\implies$ **PASSES**.

4. Test T4: compute_delivery_fee(5, 6, False)

- Base fee: 3.0; Distance surcharge: $5 \times 0.8 = 4.0 \implies \text{fee} = 7.0$
- Weight surcharge: $6 > 5 \implies \text{fee}+ = (6 - 5) \times 1.5 = 1.5 \implies \text{fee} = 8.5$
- Express: False; Cap: $8.5 \leq 50$. Return value: **8.5**
- Assertion: `assert 8.5 == 8.5` $\implies$ **PASSES**.

Pass Rate Calculation:

$$\text{Pass Rate} = \frac{\text{Number of Passing Tests}}{\text{Total Generated Tests}} = \frac{3}{4} = 75\% \quad (12)$$

Part b) Mutation Score Analysis

A mutant is **killed** if at least one passing test case in the suite fails on the mutated code:

1. Mutant 1 (Line 5: fee += distance_km * 0.08):

- On Test **T3** ($d = 60, w = 1$): $\text{fee} = 3.0 + 60 \times 0.08 = 3.0 + 4.8 = 7.8$. Since $7.8 \leq 50$, the returned fee is 7.8. The test asserts `== 50.0`, which fails!
- On Test **T4** ($d = 5, w = 6$): $\text{fee} = 3.0 + 5 \times 0.08 + 1.5 = 4.9 \neq 8.5$. Assertion fails!
- **Result:** Mutant 1 is **KILLED**.

2. Mutant 2 (Line 7: if weight_kg >= 5: instead of > 5):

- The only input where the branch predicate differs is when `weight_kg = 5.0`.
- However, for `weight_kg = 5.0`, the surcharge calculation evaluates to:

$$\text{fee}+ = (5 - 5) \times 1.5 = 0.0$$

Adding 0.0 leaves `fee` completely unchanged. Thus, the mutant behaves identically to the original function for **all possible real inputs** $\implies$ This is an **Equivalent Mutant**.

- Furthermore, none of the test cases has `weight_kg = 5` (weights are 8, 2, 1, 6).
- **Result:** Mutant 2 **SURVIVES**.

3. Mutant 3 (Line 10: if fee >= 50: fee = 50 instead of > 50):

- When $\text{fee} < 50$, neither branch triggers.
- When $\text{fee} > 50$, both assign $\text{fee} = 50$.
- When $\text{fee} == 50$, the mutant executes $\text{fee} = 50$, preserving the value of 50.
- Thus, this mutant also produces identical output to the original program under all inputs $\implies$ **Equivalent Mutant**.
- **Result:** Mutant 3 **SURVIVES**.

Mutation Score Calculation:

- **Raw Mutation Score:**

$$MS_{\text{raw}} = \frac{\text{Mutants Killed}}{\text{Total Mutants}} = \frac{1}{3} \approx \mathbf{33.33\%} \quad (13)$$

- **Effective Mutation Score (excluding equivalent mutants):**

$$MS_{\text{effective}} = \frac{\text{Mutants Killed}}{\text{Total Mutants} - \text{Equivalent Mutants}} = \frac{1}{3 - 2} = \frac{1}{1} = \mathbf{100\%} \quad (14)$$

Solution to Question 11: Multi-Agent Architecture for Use Case Modeling

1. Architecture Overview and Workflow Orchestration

We implement a **Sequential Pipeline Multi-Agent Architecture** consisting of three specialized LLaMA agents. A sequential pipeline is selected over a router or parallel architecture because requirements engineering exhibits strict unidirectional data dependencies:

$$\text{REQS} \longrightarrow \text{Agent 1: Actor Identification} \xrightarrow{\text{Actors}} \text{Agent 2: Use Case Narrative} \xrightarrow{\text{Actors} + \text{Use Cases}} \text{Agent 3: UML Diagram Generator}$$

2. Prompt Engineering Techniques Employed

1. **Persona and Role Definition:** Explicitly assigns software engineering archetypes (Requirements Modeler, Functional Specification Analyst, UML Systems Architect) to establish domain competence.
2. **Task Decomposition:** Deconstructs monolithic use case modeling into three decoupled, manageable cognitive stages.
3. **Few-Shot In-Context Demonstrations:** Provides high-quality canonical examples establishing syntactic patterns and edge-case handling.
4. **Constrained Structured Output Formatting:** Enforces strict JSON schemas and valid PlantUML grammar to ensure automated downstream parsing without regex brittleness.
5. **Chain-of-Thought (CoT) Grounding:** Instructs agents to articulate extraction reasoning prior to emitting the structured payload.

3. Comprehensive Production-Grade Agent Specifications

Agent 1: Actor Identification Agent

- **Role:** Senior Requirements Engineer & Domain Modeler.
- **Input:** Natural language requirements string {REQS}.
- **Expected Output:** Strict JSON array of actors detailing role, type, and generalization hierarchy.

```

1 [SYSTEM MESSAGE]
2 You are a Principal Requirements Engineer specializing in UML 2.5 domain
  modeling.
3 Your objective is to analyze a natural language requirements document (REQS)
  and extract all system ACTORS.
```

```

4  An actor is an external entity (human role or external software/hardware
    system) that interacts directly with the system under design.
5
6  Follow these strict modeling rules:
7  1. Identify primary actors (users initiating use cases) and
    secondary/supporting actors.
8  2. Differentiate between human users and external automated systems.
9  3. Identify actor inheritance/generalization relationships (e.g., Teacher is
    an Employee).
10 4. Output MUST strictly adhere to the provided JSON schema. No
    conversational prose.
11
12 [CONTEXT & FEW-SHOT EXAMPLES]
13 Example Input REQS:
14 "The hospital system allows staff members to view rosters. Nurses can
    additionally administer medications."
15 Example Output JSON:
16 {
17   "actors": [
18     {
19       "id": "ACT_01",
20       "name": "Staff Member",
21       "type": "Human",
22       "role": "General employee in the hospital",
23       "parent_actor": null
24     },
25     {
26       "id": "ACT_02",
27       "name": "Nurse",
28       "type": "Human",
29       "role": "Clinical healthcare provider",
30       "parent_actor": "Staff Member"
31     }
32   ]
33 }
34
35 [INPUT]
36 Analyze the following system requirements:
37 REQS: ""{REQS}""
38
39 [EXPECTED OUTPUT FORMAT]
40 Emit ONLY a valid JSON object matching this schema:
41 {
42   "actors": [
43     {
44       "id": "ACT_XX",
45       "name": "string",
46       "type": "Human | ExternalSystem",
47       "role": "string",
48       "parent_actor": "string or null"
49     }
50   ]
51 }

```

Listing 1: Full Prompt Specification for Agent 1 (Actor Identification)

Agent 2: Use Case Narrative & Description Agent

- **Role:** Functional Specification Analyst.
- **Inputs:** Natural language string {REQS} and identified {ACTORS_JSON} from Agent 1.
- **Expected Output:** JSON list of use cases structured using Cockburn's specification format.

```

1 [SYSTEM MESSAGE]
2 You are a Lead Functional Analyst. Your task is to identify and elaborate
   all Use Cases from the provided system requirements (REQS) and the
   validated list of system ACTORS.
3 A Use Case represents a discreet, goal-oriented interaction between an actor
   and the system.
4
5 For each use case, you must determine:
6 1. Primary Actor.
7 2. Goal and brief description.
8 3. Main Success Scenario (step-by-step narrative).
9 4. Inter-use-case dependencies:
10 - <<include>>: mandatory shared behavior.
11 - <<extend>>: optional or conditional extension behavior.
12 - Generalization: specialized variations of an abstract use case.
13
14 [CONTEXT & FEW-SHOT EXAMPLES]
15 Example Input REQS: "Users log into their portal to view grades."
16 Example Output JSON:
17 {
18   "use_cases": [
19     {
20       "id": "UC_01",
21       "name": "View Grades",
22       "primary_actor": "User",
23       "preconditions": "User is authenticated",
24       "main_scenario": ["Actor selects grades view", "System displays grade
25         report"],
26       "includes": ["UC_00_Login"],
27       "extended_by": [],
28       "specializes": null
29     }
30   ]
31 }
32 [INPUTS FROM PIPELINE]
33 Requirements: "{REQS}"
34 Validated Actors: {ACTORS_JSON}
35
36 [EXPECTED OUTPUT FORMAT]
37 Emit ONLY a valid JSON object:
38 {
39   "use_cases": [
40     {
41       "id": "UC_XX",
42       "name": "string (Verb + Noun)",
43       "primary_actor": "string (must match an actor name)",
44       "description": "string",
45       "includes": ["string"],
46       "extended_by": ["string"],
47       "specializes": "string or null"
48     }
49   ]
50 }

```

Listing 2: Full Prompt Specification for Agent 2 (Use Case Narrative)

Agent 3: UML Use Case Diagram Architect Agent

- **Role:** Software Architecture & Diagram Tool Specialist.
- **Inputs:** {ACTORS_JSON} from Agent 1 and {USE_CASES_JSON} from Agent 2.

- **Expected Output:** Syntactically flawless PlantUML script modeling the complete use case diagram.

```

1 [SYSTEM MESSAGE]
2 You are an Expert UML Software Architect. Your task is to synthesize the
   extracted ACTORS and USE CASES into a clean, complete, syntactically
   valid PlantUML diagram specification.
3
4 Rules for PlantUML Generation:
5 1. Wrap all diagram code strictly within @startuml and @enduml.
6 2. Group all use cases inside a 'rectangle SystemBoundary { ... }'.
7 3. Render actors using the 'actor' keyword outside the boundary.
8 4. Render actor generalization hierarchies using 'ChildActor --|>
   ParentActor'.
9 5. Model relationships between actors and use cases with solid arrows:
   'Actor --> (UseCase)'.
10 6. Model <<include>> relationships with dashed arrows: '(UC1) .> (UC2) :
   <<include>>'.
11 7. Model <<extend>> relationships with dashed arrows: '(ExtensionUC) .>
   (BaseUC) : <<extend>>'.
12 8. Model use case specialization with generalization arrows:
   '(SpecializedUC) -up-|> (BaseUC)'.
13
14 [INPUTS FROM PIPELINE]
15 Validated Actors: {ACTORS_JSON}
16 Validated Use Cases: {USE_CASES_JSON}
17
18 [EXPECTED OUTPUT FORMAT]
19 Emit ONLY the raw PlantUML code block. No explanations or introductory text.

```

Listing 3: Full Prompt Specification for Agent 3 (UML Diagram Architect)

4. Exact PlantUML Diagram Output Corresponding to the Exam Asset

When executing Agent 3 on the requirements depicted in Asset `page_7_img_0.png`, the agent outputs the following PlantUML specification:

```

1 @startuml
2 left to right direction
3 skinparam packageStyle rectangle
4 skinparam monochrome false
5 skinparam shadowing false
6
7 actor "Employee" as Employee
8 actor "Teacher" as Teacher
9
10 Teacher --|> Employee
11
12 rectangle SystemBoundary {
13     usecase "View profile" as UC_ViewProfile
14     usecase "Assign grade" as UC_AssignGrade
15     usecase "Login" as UC_Login
16     usecase "Upload CSV with grades" as UC_UploadCSV
17     usecase "Manually assign grade" as UC_ManualGrade
18     usecase "Add comment" as UC_AddComment
19
20     Employee --> UC_ViewProfile
21     Teacher --> UC_AssignGrade
22
23     UC_ViewProfile .> UC_Login : <<include>>
24     UC_AssignGrade .> UC_Login : <<include>>
25

```

```
26      UC_AddComment .> UC_AssignGrade : <<extend>>
27
28      UC_UploadCSV -up-|> UC_AssignGrade
29      UC_ManualGrade -up-|> UC_AssignGrade
30  }
31  @enduml
```

Listing 4: Synthesized PlantUML Output for Exam Use Case Diagram